

MPLADS AI RISK INTELLIGENCE PLATFORM

SIH26102 - MASTER PROJECT CONTEXT & BUILD HANDOFF

For Codex, ChatGPT Work, and the 6-member SIH team

PURPOSE OF THIS DOCUMENT

This is the consolidated source of truth for continuing the project in a new Codex or Work conversation. It is not a raw chat transcript. It captures the decisions, reasoning, architecture, data, algorithms, stack, constraints, team ownership, implementation order, testing strategy, deployment plan, demo story, and unresolved items established during project planning. New agents should preserve these decisions unless the team explicitly changes them.

CURRENT BUILD STATE

Planning and architecture are complete enough to begin implementation, but implementation has NOT started. Do not generate the whole project at once. The user wants guided, module-by-module development from an empty repository through deployment. Do not start coding until the user explicitly says START.

Problem statement

SIH26102 - Development of an AI-powered system to detect anomalies, fraud, and inefficiencies in MPLAD Scheme implementation regd.

Organization: Ministry of Statistics and Programme Implementation (MoSPI) | Department: Data Informatics & Innovation Division (DIID) | Category: Software | Theme: Smart Automation

NON-NEGOTIABLE LANGUAGE

An anomaly is an unusual pattern, not proof of fraud. The prototype should identify high-risk / suspicious / non-compliant / inefficient cases that require authorized human verification. It must not automatically accuse an MP, agency, vendor, or officer of fraud, block payments, reject works, or make final administrative decisions.

0. How a new Codex / Work agent should use this handoff

1. Treat Section 2 (Frozen decisions) as the current source of truth.
2. Treat Section 5 (Data contract) and Section 9 (Implementation contracts) as coding boundaries.
3. Do not reintroduce PostgreSQL/SQL into the MVP unless the user explicitly changes the decision.
4. Do not expose 07_anomaly_ground_truth.csv as a production/model feature. It is evaluation-only.
5. Do not train Grok on the CSVs or use Grok as the main anomaly/fraud detector.
6. Build one module at a time, run tests, review outputs, then continue.
7. When a requirement is unclear, inspect the actual CSV headers/data or ask the user; do not invent unavailable official fields.
8. Keep the core detection system functional even if the Grok API is unavailable.
9. Before implementing any compliance rule, verify it against the current official MPLADS guideline; do not encode remembered wording as law.
10. Preserve auditability: every alert should expose source evidence, detector/model/rule version, and human action.

Document map

Section	What it contains
1	Problem understanding and domain model
2	Frozen decisions and decision history
3	System objectives / success criteria
4	Stakeholders and workflow context
5	Data inventory, schemas and data rules
6	19-page architecture and box meanings
7	Algorithms, models, features and risk logic
8	Final CSV-first technology stack and libraries
9	Shared data/API contracts and folder structure
10	5-developer ownership and presenter role
11	Implementation sequence: pin-to-deploy
12	Testing, evaluation and leakage prevention
13	Frontend, dashboards and human workflow
14	RAG/Grok design and fallback
15	Security, governance, audit and limitations
16	Deployment and demo reliability
17	Judge demo story and pitch language
18	What NOT to build / overengineering guardrails
19	Unresolved items and verification checklist
20	Ready-to-paste Codex / Work boot instructions
21	Glossary and final no-drift checklist

1. Problem understanding and domain model

1.1 What SIH26102 is asking for

Build an AI-powered monitoring and analytics platform for MPLADS that can analyze sanctions, expenditures, cost estimates, work progress, payments, completion and asset creation; detect trends, anomalies, irregularities, potential fraud indicators, cost overruns, duplicate works, delayed projects and deviations from norms; generate risk-based alerts, predictive insights, automated compliance monitoring, trend analysis, early warnings and dashboards for the named authorities.

1.2 Core interpretation

- Anomaly = unusual pattern that deserves attention.
- Potential fraud / misuse = a high-risk pattern that requires authorized investigation; the AI cannot establish intent from data alone.
- Inefficiency = poor use of time, money or resources; it may exist without fraud.
- Non-compliance = a process/rule condition appears not to be satisfied.
- Success = prioritize the most review-worthy works, show exact evidence, warn early, explain clearly and preserve an audit trail - not merely claim a high accuracy percentage.

1.3 Product position

The proposed platform is a read-mostly intelligence and decision-support layer around the MPLADS lifecycle. It should complement the existing operational system (including eSAKSHI/current official workflows) rather than attempt to rebuild government transaction execution in the hackathon prototype.

1.4 Known scheme context from project research - verify before production rule encoding

- MPs recommend eligible developmental works; the MP is not the executing agency and does not personally receive MPLADS funds.
- District Authorities scrutinize/sanction/reject work recommendations, estimates and eligibility and select/coordinate implementing agencies according to the applicable process.
- Implementing Agencies execute sanctioned works and follow procurement/technical/financial processes; vendors receive authorized payments.
- State Nodal Authorities coordinate and monitor at state/UT level; MoSPI provides national administration/oversight.
- Current terminology may distinguish Nodal District Authority and Implementing District Authority; use the current official guideline when coding role/rule details.
- eSAKSHI became the newer operational platform/fund-flow context from 1 Apr 2023; do not bypass CAPTCHA or anti-bot protections. Use authorized exports/APIs or provided datasets.

VERIFY, DO NOT BLINDLY CODE

The project research also identified an annual MPLADS entitlement of Rs. 5 crore per MP and revised post-2023 fund-flow concepts. Treat these as domain context, but verify all monetary/process rules against the current official guideline before encoding compliance logic.

2. Frozen decisions and project decision history

2.1 Fresh-start decision

The team explicitly reset earlier solution choices and re-designed the solution from first principles. Older architecture/PDF ideas may be consulted only as reference; the current 10/10 lifecycle-aware architecture is the source of truth.

2.2 Architecture decisions that are currently frozen

- Lifecycle-aware intelligence is mandatory: Pre-Sanction, Execution, Completion + Asset, plus Cross-Project Trends.
- Known rules use deterministic logic; unusual patterns use statistics/unsupervised ML; predictable historical outcomes use supervised ML; duplicate detection uses semantic + geo + amount + time evidence; Grok is explanation/RAG only.
- Risk fusion must be explainable and produce review priority (Low/Medium/High/Critical), not a fraud probability.
- Human verification is mandatory before final disposition.
- Core detection must work without Grok/API availability.
- Offline training/calibration is separate from runtime scoring.
- 07_anomaly_ground_truth.csv is evaluation-only and never a production feature.
- Data quality must influence confidence; stale/missing data must not silently become wrongdoing evidence.

2.3 Major stack decision changed during planning

FINAL MVP DATA DECISION

NO SQL DATABASE for the hackathon prototype. The MVP will retrieve and process data directly from CSV files using Pandas. PostgreSQL/PostGIS/pgvector were considered earlier, then intentionally removed to reduce infrastructure and integration work.

Implications: keep separate DataFrames for one-to-many tables, create a one-row-per-work feature DataFrame, save processed results as CSV/JSON/JSONL, keep embeddings in NumPy files (or optional FAISS), and load data/models into FastAPI memory at startup.

2.4 Things that are not finalized

- Team name is not formally frozen. Suggested names included Nirikshak AI, CivicGuard AI and VigilantX.
- Exact numeric risk-fusion weights and alert thresholds are not frozen; they must be tuned/validated and remain explainable.
- Exact current xAI/Grok model identifier should be verified at implementation time. Planning assumed Grok via xAI API, previously discussed as grok-4.6.
- Exact official schemas for the two supplied files "Allocated Limit for Honble MPs.csv" and "Amount consented for Calamity.csv" were not inspected in the available runtime; never invent their headers.
- Exact compliance rules/thresholds must be derived from the current official MPLADS guideline, not remembered text.

3. System objectives, outputs and success criteria

3.1 What the prototype must demonstrate

- Create a trusted lifecycle profile for every work.
- Detect preventable risk before or during execution, not only after completion.
- Identify cost anomalies, duplicate/overlap candidates, payment anomalies, fund-vs-progress mismatches, delay risk, cost-overflow risk, compliance issues and completion/asset inconsistencies.
- Identify district/state/sector/agency/vendor/time trends and hotspots.
- Combine signals into an explainable review-priority score.
- Ground natural-language explanations in the actual detector evidence and relevant official guideline context.
- Route problem-specific case cards to role-appropriate dashboards.
- Allow authorized human review, resolution/escalation/false-positive handling and audit logging.

3.2 What the prototype must NOT claim

- "Fraud detected" as an autonomous conclusion.
- "97% accuracy" or any metric not actually measured on held-out data.
- That Grok was trained on MPLADS CSVs.
- That synthetic benchmark performance equals real-world fraud-detection performance.
- That missing records automatically mean misuse.
- That one entity is suspicious merely because it is associated with other risky works.

3.3 Operationally meaningful evaluation

In addition to standard metrics where labels exist, evaluate the quality of the ranked investigation queue. If an officer can inspect only the top 50 or top 100 works, the system should maximize the usefulness of those top-K cases while keeping false-alert burden manageable.

4. Stakeholders, role views and decision boundary

Stakeholder	Primary prototype view / responsibility
MP	Visibility into recommended works, status, major delay/cost/completion risks; not an investigation authority by default.
District Authority	Operational case queue, detailed evidence, duplicate comparisons, progress/payment timelines, verification actions.
State Nodal Authority	District comparisons, unresolved high-risk cases, implementation trends and concentrations.
MoSPI	National risk queue, state/district/sector benchmarks, systemic trends/hotspots and drill-down.
Authorized reviewer	Reviews evidence, seeks clarification/inspection, records disposition; final decision remains human.

Decision boundary

AI STOPS HERE

AI ranks, predicts, checks, retrieves and explains. Authorized officials verify, interpret administrative context and decide the final disposition. Human actions are recorded for audit and later governed model improvement.

5. Data inventory, schemas and non-negotiable data rules

5.1 Prototype files currently available

File	Rows	Purpose
00_manifest.csv	9	Dataset manifest
01_mp_master.csv	36	MP / allocation context
02_agencies_vendors.csv	108	Implementing agency / vendor master
03_works.csv	3,000	Core work lifecycle
04_payments.csv	9,727	Payment transaction history
05_progress.csv	14,902	Progress time series
06_assets_compliance.csv	1,929	Completion, asset and compliance fields
07_anomaly_ground_truth.csv	3,000	Synthetic evaluation labels ONLY
08_data_dictionary.csv	120	Field definitions / preferred sources
09_source_acquisition_plan.csv	9	Data acquisition / fallback plan

Two additional official/user-supplied files were identified: "Allocated Limit for Honble MPs.csv" and "Amount consented for Calamity.csv". Their exact headers/rows were not inspected in the current runtime. Treat them as authoritative allocation/calamity replacements once actual fields are inspected.

5.2 Exact prototype headers

01_mp_master.csv

mp_id, mp_name, house_type, tenure, state_code, state_name, constituency, nodal_district, annual_entitlement_inr, allocated_limit_inr, calamity_consent_amount_inr, record_effective_date, source_type, source_reference, synthetic_marker

02_agencies_vendors.csv

entity_id, entity_type, entity_name, state_name, district, registration_or_office_code, vendor_category, active_status, source_type, synthetic_marker

03_works.csv

work_id, mp_id, house_type, tenure, state_name, constituency, nodal_district, recommendation_date, recommendation_type, sector, sub_sector, work_description, work_location_state, district, block, village, latitude, longitude, recommended_amount_inr, technical_estimate_amount_inr, sanctioned_amount_inr, sanction_date, sanction_status, implementing_agency_id, expected_start_date, expected_completion_date, actual_start_date, current_status, current_physical_progress_pct, current_expenditure_inr, duplicate_group_reference, source_type, source_reference, synthetic_marker

04_payments.csv

payment_id, work_id, vendor_id, payment_request_date, authorization_date, payment_release_date, payment_amount_inr, payment_stage, invoice_number, cumulative_expenditure_inr, payment_status, payment_mode, pfms_reference, authorized_by_agency_id, concurrence_authority, is_final_payment, source_type, synthetic_marker

05_progress.csv

progress_id, work_id, report_date, physical_progress_pct, financial_progress_pct, expected_progress_pct_by_date, execution_stage, reported_status, reported_by_agency_id, photo_reference, geo_latitude, geo_longitude, remarks, source_type, synthetic_marker

06_assets_compliance.csv

work_id, completion_date, completion_marked_date, final_expenditure_inr, utilization_certificate_status, utilization_certificate_date, asset_id, asset_type, user_agency, asset_register_entry, handover_date, public_use_date, audit_status, audit_date, audit_observation_category, compliance_status, maintenance_agency, completion_photo_reference, source_type, synthetic_marker

07_anomaly_ground_truth.csv - EVALUATION ONLY

work_id, injected_anomaly_count, injected_anomaly_types, expected_risk_score_0_100, expected_risk_class, duplicate_group_reference, label_usage_note, synthetic_marker

LABEL LEAKAGE RULE

Never use injected_anomaly_count, injected_anomaly_types, expected_risk_score_0_100, expected_risk_class, or any ground-truth-derived field as an input feature. This file exists only to evaluate the synthetic prototype.

5.3 Synthetic-data status

The operational prototype dataset was intentionally synthetic. Operational rows include synthetic_marker=True. The source acquisition plan says the official supplied allocation/calamity files should replace corresponding synthetic demo values when their fields are available. Do not present synthetic rows as live government records.

5.4 CSV-first in-memory design**IN-MEMORY DATAFRAMES**

df_mp
df_entities
df_works
df_payments
df_progress
df_assets

One-row-per-work derived table:

df_project_features

IMPORTANT JOIN RULE

Do NOT blindly merge works x payments x progress into one giant DataFrame. A work with 5 payments and 8 progress updates can become 40 rows and corrupt sums. Aggregate one-to-many tables to work-level features or access them separately by work_id.

5.5 Canonical project profile contract

CONCEPTUAL PROJECT PROFILE

```
{
  "work_id": "W152",
  "work": {...},
  "mp_context": {...},
  "agency": {...},
  "payments": [...],
  "progress": [...],
  "asset": {...},
  "data_quality": {...},
  "features": {...}
}
```

6. Current 19-page architecture - source of truth

MASTER FLOW

Data Sources -> Trusted Project Record -> Feature Store -> Offline Training/Calibration -> Lifecycle-Aware Detection -> Cross-Project Trends -> Explainable Risk Fusion -> Guideline Retrieval -> Grok Grounded Explanation -> Problem-Specific Alerts -> Role-Based Dashboards -> Human Verification -> Governed Feedback

6.1 Exact page names

Page	Name
1	00 - 10/10 MASTER FLOW
2	Data Sources & Prototype Schema
3	Trusted Data Foundation
4	Feature Store
5	Offline Training / Calibration
6	Validation & Threshold Tuning
7	Approved Model / Rule Registry
8	Runtime / Batch Scoring
9	Pre-Sanction Intelligence
10	Execution Intelligence
11	Completion + Asset Intelligence
12	Cross-Project Trend Intelligence
13	MPLADS Compliance Rule Pack
14	Explainable Risk Fusion
15	Grok API - Grounded Explanation / RAG
16	Problem-Specific Alert Outputs
17	Role-Based Decision Support
18	Human Verification + Governed Feedback
19	Judge Demo Story

6.2 Page 1 - Master flow, box by box

MP + Allocation

Administrative/allocation context from MP master and official allocation/calamity files. Context, not wrongdoing evidence.

Work Lifecycle

Core work anchor: recommendation, estimate, sanction, location, agency, status, expenditure, expected/actual dates.

Payments

Transaction-level timing, amount, vendor and cumulative expenditure.

Progress

Time series of physical/financial progress and expected progress.

Completion / Asset

Completion date, final expenditure, UC, asset, handover and audit/closure evidence.

MPLADS Rules

Official guidelines/norms used for deterministic compliance checks and RAG grounding.

Ingest + Validate

Schema, dates, money, missingness, impossible chronology, provenance.

Link Lifecycle

Connect MP <-> work <-> agency <-> vendor <-> payments <-> progress <-> completion/asset.

Unified Project Profile

One versioned work timeline plus data-quality score.

Feature Store

Financial, temporal, payment, text, geo, entity and closure features.

Time-Aware Train/Validation/Test

Learn from older records, tune on later records, final test on unseen later records; prevent leakage.

Anomaly Baseline

Isolation Forest + robust peer statistics.

Delay / Overrun

Supervised tree-based prediction with calibration and explainability.

Duplicate Thresholds

Semantic + geo + amount + time similarity.

Rule Pack

Versioned deterministic MPLADS conditions.

Validate + Tune

Precision/recall/F1 where valid, false alerts, top-K queue quality, thresholds.

Version, threshold, metrics, rollback and reproducibility.

New / Updated Event

Recommendation/sanction/payment/progress/completion change triggers scoring, not retraining.

Rebuild Current Features

Recompute affected work features using same definitions as training.

Pre-Sanction Intelligence

Peer cost benchmark + duplicate candidate + eligibility/compliance; prevent avoidable risk before spending.

Execution Intelligence

Payment/utilization anomaly + fund-vs-progress + delay + overrun + vendor/agency context.

Completion + Asset Intelligence

Completion consistency + final finance + post-completion payment + asset/UC/handover.

Cross-Project Trends

State/district/sector/agency/vendor/year-quarter baselines, change detection and hotspot concentration.

Explainable Risk Fusion

Severity x confidence x data quality x lifecycle relevance; preserve component evidence.

LOW / MEDIUM / HIGH / CRITICAL

Operational verification priority tiers, not guilt levels.

Guideline Retrieval

Retrieve only relevant official guideline passages.

Grok Grounded Explanation

Why flagged + evidence + guideline context + what to verify next; not core detection.

Problem-Specific Alerts

Cost, duplicate, delay, fund-progress, payment, compliance, asset/closure, trend/hotspot.

Role-Based Dashboards

MP, District, State, MoSPI views.

Authorized Human Verification

Review, clarify, inspect, resolve/escalate/false-positive; human decides.

Governed Feedback

Reviewed outcomes improve later model/rule versions after offline validation, not instant self-learning.

6.3 Pages 2-19 - what each page proves

Page	Purpose / judge question
2 Data Sources	Where does the data come from and how can official sources replace prototype rows?
3 Trusted Foundation	Can the AI trust the data?
4 Feature Store	What exactly enters models?

5 Training	How are the intelligence engines built?
6 Validation	How do we know they are useful?
7 Registry	Can a past alert be reproduced/rolled back?
8 Runtime	What happens when new data arrives?
9 Pre-Sanction	How does the system prevent risk before spending?
10 Execution	How does it monitor money, work and time together?
11 Completion	How does monitoring continue through asset/closure?
12 Trends	How do we detect systemic patterns?
13 Compliance	Which checks are deterministic and guideline-based?
14 Risk Fusion	How do multiple signals become one review priority?
15 Grok/RAG	What role does the LLM play, and what does it not do?
16 Alerts	How is evidence presented differently for each problem?
17 Role Support	Who sees what?
18 Human + Feedback	Where does human authority begin and how does learning remain governed?
19 Demo	How do we tell the full evidence-to-decision story in 3-5 minutes?

7. Algorithms, models, feature engineering and risk logic

7.1 Final method map

Problem	Method	Exact algorithm / model
Peer cost anomaly	Robust statistics	Median + percentiles + IQR + Median Absolute Deviation (MAD)
General unusual patterns	Unsupervised ML	Isolation Forest (scikit-learn)
Payment anomaly	Hybrid	Rules + rolling/payment features + peer statistics + Isolation Forest
Fund vs physical progress	Deterministic analytics	Financial-progress formula + percentage-point gap + persistence over time
Delay prediction	Supervised ML	XGBoost Classifier; Random Forest Classifier baseline; calibrated probabilities
Cost-overrun prediction	Supervised ML	XGBoost Classifier; optional XGBoost Regressor for expected final expenditure
Duplicate work text	NLP embeddings	sentence-transformers/all-MiniLM-L6-v2
Duplicate similarity	Vector similarity	Cosine similarity + geo distance + amount similarity + time similarity
Vendor concentration	Statistics	HHI / concentration shares; context only
Compliance	Deterministic	Versioned Python IF/THEN rule engine
Completion / asset	Deterministic + spatial	Consistency, reconciliation, post-completion timing, evidence completeness, geo checks
Trends	Time-series statistics	Rolling baselines, robust z-score, EWMA, percentage change; optional PELT change point
Hotspots	Aggregation / optional clustering	Geographic/group concentration; optional DBSCAN
Model explainability	XAI	SHAP for tree-based predictive models
Risk fusion	Transparent scorecard	Weighted normalized detector scores + confidence + data quality + lifecycle relevance
Guideline retrieval	Semantic retrieval	PyMuPDF chunks + SentenceTransformer embeddings + cosine / NearestNeighbors; optional FAISS
Natural-language case explanation	LLM + RAG	Grok via xAI API; planned model ID should be verified at implementation time

7.2 Core formulas / features

FINANCIAL VS PHYSICAL

Source-of-truth handoff for Codex / Work - prototype decisions as of 01 Sep 2026

```
financial_progress_pct = current_expenditure_inr / sanctioned_amount_inr * 100
financial_physical_gap_pp = financial_progress_pct - current_physical_progress_pct
```

- Cost ratios: estimate/recommended, sanction/estimate, expenditure/sanction, final expenditure/sanction.
- Temporal features: recommendation-to-sanction days, sanction-to-start days, project age, elapsed-time ratio, days overdue, days since last progress update.
- Progress features: latest physical progress, expected progress, actual-vs-expected gap, recent velocity/slope, stagnation duration.
- Payment features: payment count, mean/median/max payment, largest-payment share, 7/30-day payment count and amount, days between payments, payment burst, vendor share, cumulative utilization.
- Duplicate features: text embedding similarity, Haversine/geodesic distance, amount similarity, date proximity, same sector/category/location context.
- Closure features: completed flag vs progress, final-expenditure reconciliation, payments after completion, UC status, asset ID/location, handover/public-use/audit completeness.
- Entity context: agency/vendor project count, delayed-work share, overrun share, payment concentration, workload; context only, never guilty by association.

7.3 Peer-cost design

Build defensible peer cohorts using sector/sub-sector, geography, time period and project scale where available. Avoid comparing unrelated work types. Use median/percentile/MAD-based deviation because robust statistics are easier to explain and less sensitive to extreme values than simple averages.

7.4 Duplicate confidence

Start with transparent prototype weights only as an initial configuration; tune them on validation evidence rather than presenting arbitrary weights as scientifically final. The output must show Work A vs Work B, semantic similarity, distance, amount difference and date difference.

7.5 Risk fusion

CONCEPTUAL FORMULA

Risk contribution is based on detector severity x detector/model confidence x data quality x lifecycle relevance. Independent strong signals agreeing can increase priority. Exact weights/thresholds remain configurable and must be documented.

Example evidence, not a frozen formula: progress mismatch high + delay probability 0.84 + payment burst high + moderate cost anomaly -> fused risk 86/100, tier HIGH, top drivers shown. The number 86 is a review priority, not an 86% probability of fraud.

8. Final CSV-first technology stack and libraries

8.1 Prototype stack summary

Layer	Technology
Language	Python 3.12+ for data/ML/backend; TypeScript for frontend
Data source	CSV files loaded directly; no SQL database in MVP
Data manipulation	pandas, numpy, scipy, pyarrow
Validation	pandera, pydantic, pydantic-settings
Dates / files	python-dateutil, openpyxl
Name matching	rapidfuzz; optional unidecode
Geo distance	geopy / Haversine math
Core ML	scikit-learn
Advanced tabular ML	xgboost
Explainability	shap
Model persistence	joblib
NLP embeddings	sentence-transformers, torch/transformers
Vector search	sklearn cosine_similarity / NearestNeighbors; optional faiss-cpu
PDF guideline extraction	PyMuPDF
LLM	Grok via xAI API / xai-sdk; verify exact model ID before implementation
Backend	FastAPI + uvicorn + pydantic
HTTP / serialization	httpx, orjson, python-multipart
Auth demo	JWT via PyJWT + pwdlib, if role login is implemented
Frontend	Next.js + React + TypeScript
UI	Tailwind CSS + shadcn/ui + lucide-react
Charts	Recharts
Maps	MapLibre GL JS
Frontend data	TanStack Query, Zod, React Hook Form, date-fns
Testing	pytest + pytest-asyncio + Playwright + Postman
Version control	Git + GitHub
Containers	Docker; local demo fallback
Secrets	.env + pydantic-settings; commit .env.example only

8.2 Python package plan

PYTHON DEPENDENCIES

pandas
 numpy
 scipy
 pyarrow
 openpyxl
 python-dateutil

pandera
pydantic
pydantic-settings
rapidfuzz
geopy
scikit-learn
xgboost
joblib
shap
sentence-transformers
torch
transformers
pymupdf
xai-sdk
fastapi
uvicorn
httpx
orjson
python-multipart
PyJWT
pwdlib
pytest
pytest-asyncio

Optional only if needed:

faiss-cpu
ruptures
mlflow
optuna
structlog

8.3 Frontend package plan

FRONTEND DEPENDENCIES

next
react
react-dom
typescript
tailwindcss
shadcn/ui
lucide-react
recharts
maplibre-gl
@tanstack/react-query
zod
react-hook-form
date-fns

8.4 Why no SQL / vector database in the MVP

The dataset is small enough (about 3,000 works, 9.7k payments and 14.9k progress records) to load into memory. Direct CSV/Pandas reduces setup and integration risk. Similarity over a few thousand work descriptions and guideline chunks can be done with NumPy/scikit-learn; FAISS is optional. Production can later migrate the same logical schema to a database without changing detector concepts.

9. Shared code contracts, outputs and repository structure

9.1 Golden integration object

SHARED CONTRACT

```
{
  "work_id": "",
  "project_profile": {},
  "features": {},
  "detector_scores": {},
  "risk": {},
  "evidence": [],
  "explanation": {},
  "case_status": ""
}
```

Member 1 fills project data/features; Member 2 fills numeric ML detector results; Member 3 fills duplicate/compliance/RAG evidence; Member 4 fills fused risk/trends; Member 5 consumes the object in the UI.

9.2 Processed artifact plan

CSV / JSON OUTPUTS

```
data/processed/
  project_features.csv
  detector_results.csv
  risk_results.csv
  duplicate_candidates.csv
  compliance_results.csv
  trend_results.csv
```

```
data/reviews/
  review_actions.json
  audit_log.jsonl
```

```
cases/
  W152.json # detailed evidence-ready case
```

9.3 FastAPI startup behavior

RUNTIME STARTUP

```
FastAPI starts
-> read CSVs once
```

- > validate / normalize
- > load processed feature/results files
- > load trained joblib models
- > load embeddings/chunks
- > keep DataFrames/models in memory
- > serve requests

Do not call `pd.read_csv` on every dashboard click. Load once at startup or through a controlled reload endpoint during development.

9.4 Suggested API surface

CONCEPTUAL API

- GET /health
- GET /works
- GET /works/{work_id}
- GET /works/{work_id}/risk
- GET /works/{work_id}/payments
- GET /works/{work_id}/progress
- GET /works/{work_id}/duplicate-candidates
- GET /works/{work_id}/explanation
- GET /alerts
- GET /trends
- GET /dashboard/mp
- GET /dashboard/district
- GET /dashboard/state
- GET /dashboard/mospi
- POST /reviews/{case_id}

9.5 Repository structure

RECOMMENDED REPOSITORY

- mplads-ai/
 - data/
 - raw/
 - processed/
 - reviews/
 - intelligence/
 - data_loader.py
 - validation.py
 - project_profile.py
 - features/
 - anomaly/
 - delay/
 - overrun/
 - payments/
 - duplicates/
 - compliance/
 - completion/

```
trends/  
risk_fusion/  
rag/  
guideline_ingestion/  
retrieval/  
prompts/  
grok/  
models/  
trained/  
configs/  
metrics/  
rules/  
backend/  
  app/  
  tests/  
frontend/  
scripts/  
tests/  
docker/  
.env.example  
.gitignore  
README.md
```

10. Team ownership - 5 developers + 1 presenter

Member	Ownership	Primary deliverable
Developer 1 - Data / Backend Foundation	CSV loading, validation, linking, unified project profile, feature tables, FastAPI data endpoints	Given a work_id, return trusted project profile + features.
Developer 2 - Core ML / Predictive Intelligence	Isolation Forest, peer stats support, payment anomaly ML, delay/overrun models, evaluation, SHAP, model files	Detector scores/probabilities + model drivers + validation metrics.
Developer 3 - Duplicate / Compliance / RAG	Embeddings, cosine + geo/amount/time duplicate logic, rule engine, guideline chunk/retrieval, Grok integration	Duplicate candidates, rule hits, retrieved clauses, grounded explanation.
Developer 4 - Lifecycle Risk / Trends	Pre-sanction/execution/completion orchestration, fund-vs-progress, closure checks, trends, risk fusion	Final risk_case: score, tier, confidence, alert types, top drivers, trends.
Developer 5 - Product / Integration	Next.js UI, case queue, work detail, charts/maps, role views, review actions, audit UI, integration/deploy	Working web prototype and judge demo.
Member 6 - Presenter	Understands all modules, maintains judge story, collects evidence/screens, rehearses demo and Q&A	3-5 minute demo + explanation; no isolated coding ownership required.

Integration rule

All developers must agree on shared Pydantic/JSON shapes before implementing independently. Do not let each member invent different field names and payload formats.

Presenter discipline

Each developer should explain their module to the presenter in five minutes using beginner language. If the presenter cannot understand a module, the judge likely will not either.

11. Pin-to-deploy implementation sequence

CODING PHILOSOPHY

Do not ask Codex to "build the entire platform" in one run. Give one bounded task, inspect changed files, run tests, understand the result, commit, then continue.

1. Phase 0 - Repository and environments

Create Git repo, branch strategy, Python environment, Node/Next environment, folders, .gitignore, .env.example, README skeleton, lint/test config. No ML yet.

2. Phase 1 - Data loader

Load all CSVs once, inspect actual headers, parse dates/numerics, keep separate DataFrames, basic required-column checks.

3. Phase 2 - Validation

Pandera/Pydantic schemas, impossible ranges/chronology, orphan IDs, missingness, staleness and provenance. Unit tests.

4. Phase 3 - Unified project profile

get_work/get_payments/get_progress/get_asset/build_project_profile; one canonical work-centric object.

5. Phase 4 - Feature engine

One-row-per-work df_project_features with financial, temporal, progress, payment, peer, text/geo placeholders, closure features.

6. Phase 5 - Peer cost / deterministic signals

Peer cohorts, median/MAD/percentile cost deviation; financial-vs-physical gap + persistence; basic closure checks.

7. Phase 6 - Anomaly / payment engine

Isolation Forest training/scoring using production-safe features only; payment feature/rule hybrid; held-out/synthetic evaluation.

8. Phase 7 - Delay and overrun models

Derive legitimate historical targets from outcome fields; time-aware split; Random Forest baseline and XGBoost; calibration; SHAP; save joblib + metrics.

9. Phase 8 - Duplicate detector

Embed work descriptions with all-MiniLM-L6-v2; cosine candidates; combine geo, amount and time; tune threshold; save candidate CSV.

10. Phase 9 - Compliance rules

Create versioned Python rule definitions only after verifying guideline requirements; output rule hits + evidence.

11. Phase 10 - Completion/asset intelligence

Completion consistency, final finance reconciliation, post-completion payments, asset/UC/handover completeness.

12. Phase 11 - Trend intelligence

District/state/sector/agency/vendor/time aggregations, rolling baseline, change measures, hotspot outputs.

13. Phase 12 - Risk fusion

Normalize signals; transparent configurable weights; confidence/data-quality/lifecycle modifiers; output risk score/tier/top drivers.

14. Phase 13 - Guideline RAG

Extract official PDF with PyMuPDF, chunk, embed, retrieve top relevant passages. Validate source references.

15. Phase 14 - Grok explanation

Structured prompt from facts + detector evidence + retrieved passages; Pydantic-validated JSON output; fallback when API unavailable.

16. Phase 15 - FastAPI

Health, works, risk, alerts, trends, explanation and review endpoints. Load data/models in memory. OpenAPI docs.

17. Phase 16 - Frontend foundation

Next.js/TypeScript/Tailwind/shadcn, navigation, shared API client/types, risk queue and work detail skeleton.

18. Phase 17 - Evidence UI

Money-vs-progress chart, payment timeline, delay drivers, peer cost, duplicate comparison, compliance clause, asset/closure checklist, trend views/maps.

19. Phase 18 - Role dashboards

MP/District/State/MoSPI scoping and case queues; role login can be simple prototype RBAC.

20. Phase 19 - Human verification / audit

Review actions, notes, disposition, review_actions.json and audit_log.jsonl; show case history.

21. Phase 20 - Integration + E2E tests

Run full flow on known synthetic cases, fix payload mismatches, Playwright demo path, API tests, model regression checks.

22. Phase 21 - Deployment

Docker/local fallback; deploy frontend/backend if desired; secure env vars; cache demo Grok explanations; rehearse offline mode.

23. Phase 22 - SIH demo freeze

Freeze data/model versions, precompute demo results, record metrics honestly, final screenshots, presenter rehearsal and contingency plan.

12. Testing, evaluation and leakage prevention

12.1 Unit tests

- CSV required columns and date parsing
- No duplicated work-level feature rows
- Financial-progress formula edge cases including zero/invalid sanction
- Payment aggregation correctness
- Progress velocity/staleness
- Peer cohort selection
- Rule conditions and versions
- Risk fusion bounds and top-driver ordering
- RAG retrieval returns source references
- API response validation

12.2 ML evaluation

- Time-aware train/validation/test split where possible
- Precision, recall, F1 and PR-AUC/ROC-AUC where labels are meaningful
- False-positive rate and confusion matrix
- Probability calibration checks for predictive models
- Top-K precision / recall for officer queue usefulness
- Inspect false positives and subgroup behavior
- Never report synthetic evaluation as real-world fraud accuracy

12.3 Synthetic ground truth

Use 07_anomaly_ground_truth.csv only after producing model/rule outputs from operational features. Join predictions to labels for evaluation. Never feed the labels back into the production feature pipeline.

12.4 Testing stack

Area	Tool
Python/unit/API	pytest, pytest-asyncio, FastAPI TestClient/httpx
Frontend E2E	Playwright
Manual/API contract	Postman / FastAPI Swagger docs
Lint/format Python	ruff + black; optional mypy
Frontend quality	ESLint + Prettier + TypeScript checks

13. Frontend, dashboards and human workflow

13.1 Core screens

- National/state overview
- Risk-ranked case queue
- Work detail / lifecycle timeline
- Financial vs physical progress chart
- Payment timeline
- Cost peer benchmark
- Delay / overrun prediction and drivers
- Duplicate side-by-side comparison + distance
- Compliance evidence / guideline context
- Asset & closure checklist
- Trend / hotspot analytics
- Grok explanation panel
- Officer review/disposition
- Audit history

13.2 Problem-specific case cards

Alert	Evidence that must be visible
Cost / Overrun	Peer median/range, project amount, deviation, predicted overrun/final cost where supported.
Duplicate	Work A vs B, descriptions, semantic similarity, geo distance, amount/date similarity.
Delay	Probability, time elapsed, progress, velocity, model drivers.
Fund vs Progress	Financial %, physical %, percentage-point gap, history/persistence.
Payment	Exact transactions/time window, burst/concentration metrics.
Compliance	Rule ID/version, field values, retrieved official clause.
Asset / Closure	Completion contradiction, final finance, post-completion payment, missing UC/asset/handover.
Trend / Hotspot	Current metric vs baseline, affected volume, drill-down works.

13.3 Human actions

- Acknowledge / open review
- Request clarification
- Send for physical/financial verification
- Resolve
- Mark false positive
- Escalate / further review
- Record confirmed issue only after authorized review establishes it

14. RAG / Grok design and fallback

14.1 Pipeline

RAG PIPELINE

Official MPLADS guideline PDF

- > PyMuPDF text extraction
- > chunk + source metadata
- > SentenceTransformer embeddings
- > NumPy / sklearn cosine retrieval (optional FAISS)
- > top relevant passages

Project facts + detector evidence + retrieved passages

- > Grok API
- > structured explanation JSON

14.2 Grok responsibilities

- Explain why the case was flagged in plain language
- Summarize exact evidence already computed
- Use retrieved official guideline context
- Generate a concise verification checklist
- State uncertainty/limitations when evidence is missing

14.3 Grok must not

- Calculate the authoritative risk score
- Invent compliance rules
- Declare fraud or guilt
- Replace deterministic calculations/models
- Receive the entire database when only a few fields/passages are needed

14.4 Structured output

LLM RESPONSE CONTRACT

```
{
  "summary": "...",
  "why_flagged": ["..."],
  "evidence": ["..."],
  "guideline_context": [{"text": "...", "source": "..."}],
  "verification_steps": ["..."],
  "limitations": "..."
}
```

14.5 Offline/API failure fallback

If Grok is unavailable, all detector scores, risk tiers, evidence, charts and review workflow must continue to work. Show a clear "AI explanation temporarily unavailable" message plus the raw detector evidence. Cache one or two demo explanations so the SIH demonstration is not internet-dependent.

15. Security, governance, auditability and limitations

15.1 Governance principles

- Risk/anomaly != fraud proof
- Human verification required
- Data quality influences confidence
- Show source evidence and model/rule version
- Keep deterministic rules separate from predictive models
- No uncontrolled online self-learning
- No guilt by association
- Do not send unnecessary personal/transaction data to an external LLM
- Do not bypass CAPTCHA/anti-bot controls on official portals

15.2 Versioning without a database

Store model/config artifacts and metadata in files: isolation_forest_v1.joblib, delay_xgb_v1.joblib, overrun_xgb_v1.joblib, thresholds_v1.json, rules_v1.json/yaml, metrics_v1.json. The detailed case output should record the versions used.

15.3 Human feedback

Review outcomes become future labels only after appropriate verification. Periodically analyze false positives/false negatives, retrain/recalibrate offline, validate the new version, then deploy with rollback. One click must never instantly change the production model.

15.4 Prototype limitations to state honestly

- Most operational data is synthetic; real-world performance must be validated on authorized official historical data.
- The two official allocation/calamity CSV schemas must be inspected before integration.
- Compliance coverage is limited to rules actually encoded and verified from current guidelines.
- Text/geo duplicate matching creates candidates, not proof of duplicate sanction/work.
- Prediction quality depends on historical outcome quality and feature availability.
- RAG explanation quality depends on accurate guideline extraction/retrieval and API availability.

16. Deployment and demo reliability

16.1 Development

- GitHub repository with develop + feature branches or small PR workflow
- Python venv/uv; Node environment
- Use .env for secrets; commit .env.example only
- FastAPI and Next.js can run locally during early integration

16.2 Deployment options

Frontend can be deployed to Vercel. FastAPI can run on a simple container platform such as Render/Railway/Cloud Run or equivalent available to the team. For SIH, a local Docker or local process fallback is more important than sophisticated cloud infrastructure.

16.3 Offline demo fallback

DEMO RESILIENCE

Laptop

- > frontend local build
- > FastAPI local server
- > CSV + joblib + embeddings on disk
- > all core risk detection works offline
- > cached Grok explanation or graceful fallback if internet is down

16.4 Docker

Docker remains useful to reproduce backend/frontend environments, but there is no database container in the MVP. Do not introduce Kubernetes, Kafka, Spark or other infrastructure unless a real need appears.

17. Judge demo story and pitch language

11. Start at the national/state view and show many works ranked by verification priority.
12. Open one deliberately strong HIGH-risk demo work.
13. Show its lifecycle: recommendation -> sanction -> payments/expenditure -> progress -> completion context.
14. Show independent evidence, e.g. 82% funds utilized vs 39% physical progress plus 76% delay risk.
15. Show peer-cost or duplicate evidence if present.
16. Open the grounded Grok explanation: why flagged, relevant guideline context, what to verify.
17. Perform an authorized officer disposition such as request verification / escalate / resolve.
18. Show audit history: source snapshot, detector/model/rule versions and officer action.

PITCH LINE

We do not merely show anomalies. We take an officer from national risk prioritization to exact evidence, guideline-grounded explanation, human verification and an auditable outcome.

30-second solution explanation

Our platform creates a unified lifecycle profile for every MPLADS work by connecting recommendation, sanction, expenditure, payments, progress, completion and asset data. Before execution we check cost, duplicates and compliance; during execution we monitor payment behaviour, financial-versus-physical progress and predict delay/overrun; at completion we verify final finance, closure and asset evidence. Cross-project analytics find district, sector, agency and time trends. The signals are fused into an explainable review priority. We retrieve relevant official MPLADS guideline context and use Grok only to explain why the case was flagged and what should be verified. The appropriate authority reviews the evidence and makes the final decision, while verified outcomes improve future governed model versions.

18. Overengineering guardrails - technologies / ideas deliberately not needed now

Do not add by default	Reason
PostgreSQL / SQL / MongoDB	CSV/Pandas is sufficient for MVP and has been explicitly chosen.
PostGIS / pgvector / Pinecone	Geo/vector workloads are small enough for geopy + in-memory embeddings/sklearn.
TensorFlow / custom neural networks	No current problem requires deep learning for tabular data.
Graph Neural Networks	Interesting research idea but unnecessary complexity for MVP.
Kafka / Spark	Data volume is tiny for these tools.
Kubernetes	No deployment requirement justifies it.
Blockchain	Does not solve the stated monitoring problem.
LangChain everywhere	Direct retrieval + SDK code is simpler and more auditable.
Multiple LLM providers	One LLM explanation layer is enough for MVP.
Live self-learning	Governance and validation risk.
Computer vision	Optional future extension; not required for core prototype.
Automatic fraud verdict / automated blocking	Administratively and technically inappropriate.

19. Unresolved items and verification-before-code checklist

- Inspect the actual two official allocation/calamity CSV files and record exact headers/types before wiring them into the loader.
- Obtain/confirm the exact current official MPLADS guideline PDF to use for compliance and RAG. Record document version/date.
- Confirm the exact xAI model ID and SDK behavior at implementation time; do not hardcode stale documentation assumptions.
- Define the exact historical target for delay (e.g., actual completion later than expected completion by what criterion) from available fields; avoid target leakage.
- Define overrun target using appropriate final/sanction/revised amounts supported by data.
- Decide peer-cohort fallback logic when there are too few comparable works.
- Decide initial risk-fusion weights and thresholds; mark them prototype-configurable and validate against the synthetic evaluation set.
- Decide whether role authentication is real login/JWT or a demo role switch; do not spend too much time on auth before core evidence works.
- Decide deployment service based on team accounts and event constraints; preserve local fallback.
- Choose/freeze team name if required for the submission/PPT.

20. Ready-to-paste startup instructions for Codex and Work

20.1 Codex repository boot instruction

PASTE INTO CODEX

We are implementing SIH26102: an AI risk-intelligence platform for MPLADS. Read the attached Master Project Context & Build Handoff PDF before making changes. Treat it as the source of truth.

Important frozen decisions:

- CSV-first MVP. NO SQL/database. Use Pandas/in-memory DataFrames and CSV/JSON/JSONL outputs.
- Do not use 07_anomaly_ground_truth.csv as model input; it is evaluation-only.
- Lifecycle-aware architecture: pre-sanction, execution, completion/asset + cross-project trends.
- Isolation Forest + robust statistics for anomaly patterns; XGBoost/Random Forest for delay/overrun; SentenceTransformers + cosine + geo/amount/time for duplicate candidates; deterministic Python rules for compliance; transparent risk fusion; Grok only for grounded RAG/explanation.
- Anomaly/risk is not proof of fraud. Human verification is mandatory.
- Core detection must work without Grok.
- Do not redesign the architecture or add infrastructure unless asked.
- Work one bounded module at a time. Run tests. Do not modify unrelated files. Explain what changed and why.

CURRENT STATE: no production code has been written yet. Do not build the whole platform in one pass. First inspect the repository/current files and propose only the exact Phase 0 setup from the handoff. Wait for approval before coding.

20.2 Work conversation boot instruction

PASTE INTO WORK

We are continuing the SIH26102 MPLADS AI Risk Intelligence project. Read the attached Master Project Context & Build Handoff PDF and use it as persistent context. Do not redesign the solution from scratch.

We are beginning guided implementation from zero code through deployment, but we will implement and verify one module at a time. The current MVP is CSV-first with no SQL database. Preserve the architecture, model choices, risk/governance language, team ownership, and implementation sequence in the PDF.

When helping us, distinguish: (1) frozen project decisions, (2) prototype assumptions, and (3) items that still need verification. Never invent missing official file fields or compliance rules. Never treat anomaly as proven fraud.

Do not start coding until we explicitly say START. First confirm that you understand the implementation sequence and wait.

20.3 Suggested Codex /init AGENTS.md principles

AGENTS.MD CORE

Project: SIH26102 MPLADS AI Risk Intelligence

- CSV-first MVP; no SQL/database unless user explicitly changes decision.
- 07_anomaly_ground_truth.csv is evaluation-only; never a feature.

Source-of-truth handoff for Codex / Work - prototype decisions as of 01 Sep 2026

- AI outputs risk/anomaly/compliance indicators requiring human verification; never autonomous fraud verdicts.
- Core detection must work without external LLM.
- Grok is RAG/explanation only.
- Keep modules small, typed and testable.
- Preserve shared schemas/contracts; do not invent incompatible JSON shapes.
- Do not change unrelated files.
- Run tests after every meaningful change.
- Do not report metrics that were not measured.
- Verify official guideline rules before encoding them.
- Never bypass CAPTCHA/anti-bot protections.
- Prefer simple, explainable methods over unnecessary complexity.

21. Beginner glossary and final no-drift checklist

21.1 Terms the team / agent must understand

Term	Meaning
Feature	A model input variable such as financial progress, project age or payment concentration.
Label / target	Known outcome a supervised model learns to predict.
Supervised learning	ML trained on examples with known outcomes, e.g. delayed yes/no.
Unsupervised learning	ML that finds patterns/outliers without a correct label for every row.
Isolation Forest	Unsupervised anomaly detector that isolates unusual records quickly.
Random Forest	Many decision trees combined into one prediction; strong baseline for tabular data.
XGBoost	Gradient-boosted decision trees; strong for structured/tabular data.
Calibration	Making probability outputs correspond more closely to observed frequencies.
SHAP	Method for explaining individual model predictions using feature contributions.
Embedding	Numeric vector representing text meaning.
Cosine similarity	Similarity measure often used for embeddings.
RAG	Retrieve trusted source passages first, then give them to an LLM for grounded generation.
LLM	Large Language Model, used here for explanation, not authoritative fraud detection.
Data leakage	Future/answer information accidentally entering training and inflating metrics.
Precision	Of flagged positives, fraction truly positive according to evaluation labels.
Recall	Of all positives, fraction found by the system.
F1	Combined precision/recall measure.
Top-K	First K highest-ranked cases; operationally useful for limited review capacity.
Risk fusion	Combining multiple detector signals into one explainable review priority.
Provenance	Where data came from and how it was transformed.
Audit trail	Record of evidence, model/rule versions and human actions.
Human in the loop	A human retains decision authority over important AI-supported outcomes.

21.2 Final no-drift checklist

[] No SQL/database in MVP unless user explicitly changes decision.

[] No ground-truth leakage.

Source-of-truth handoff for Codex / Work - prototype decisions as of 01 Sep 2026

- [] No fraud verdict language.
- [] No Grok-as-detector redesign.
- [] No one-model-solves-everything claim.
- [] No fabricated official fields/rules.
- [] No unnecessary deep learning/infrastructure.
- [] No blind many-to-many CSV merge.
- [] No model metric claims before measurement.
- [] No API dependency that makes core demo fail.
- [] Every alert exposes evidence and human next step.
- [] Every module follows lifecycle stage and shared contract.
- [] Every meaningful code change is tested before proceeding.

FINAL SOURCE-OF-TRUTH SENTENCE

Build an explainable, lifecycle-aware MPLADS risk-intelligence prototype that converts CSV lifecycle records into evidence-backed review priorities, predictive early warnings, grounded guideline explanations and human-verifiable case workflows - while remaining simple enough for a five-developer SIH team to understand, test, demo and defend.